

**SIMATS ENGINEERING
ASSESSMENT TOOL 1**

COURSE CODE: CSA14

COURSE NAME: COMPILER DESIGN

COURSE OUTCOME COVERED

CO3: *Analyze syntax-directed translation method using synthesized and inherited attributes to generate a Parser Tree. (BL4)*

Assessment Tool Weightage: 40%

**Duration :60 Mins
On Class**

QUESTIONS:5 Total Marks:50

Annexure A

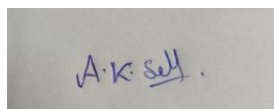
EXPERIMENTS

Code of Conduct:

*I A K Selva Prabhu(192421224) (Name / Reg No)
certify that this submission is my original work and that I have adhered to the
guidelines specified for this assessment.*

*I understand that any violation of academic integrity rules will result in
disciplinary action.*

Signature of the student:

A rectangular box containing a handwritten signature in blue ink that reads "A.K. Selva".

Annexure A

EXPERIMENTS

1 Arithmetic Expression Evaluation

Aim

To implement a syntax-directed definition for evaluating an arithmetic expression using the given grammar.

Algorithm

1. Read the arithmetic expression values.
2. Evaluate multiplication first.
3. Add the result to the first number.
4. Display the attribute values.
5. Display the final value.

C Program

```
#include <stdio.h>
```

```
int main() {
```

```
    int num1 = 3, num2 = 5, num3 = 2;
```

```
int t, e;
```

```
printf("Expression: 3 + 5 * 2\n");
```

```
t = num2 * num3;
```

```
printf("T = %d * %d = %d\n", num2, num3, t);
```

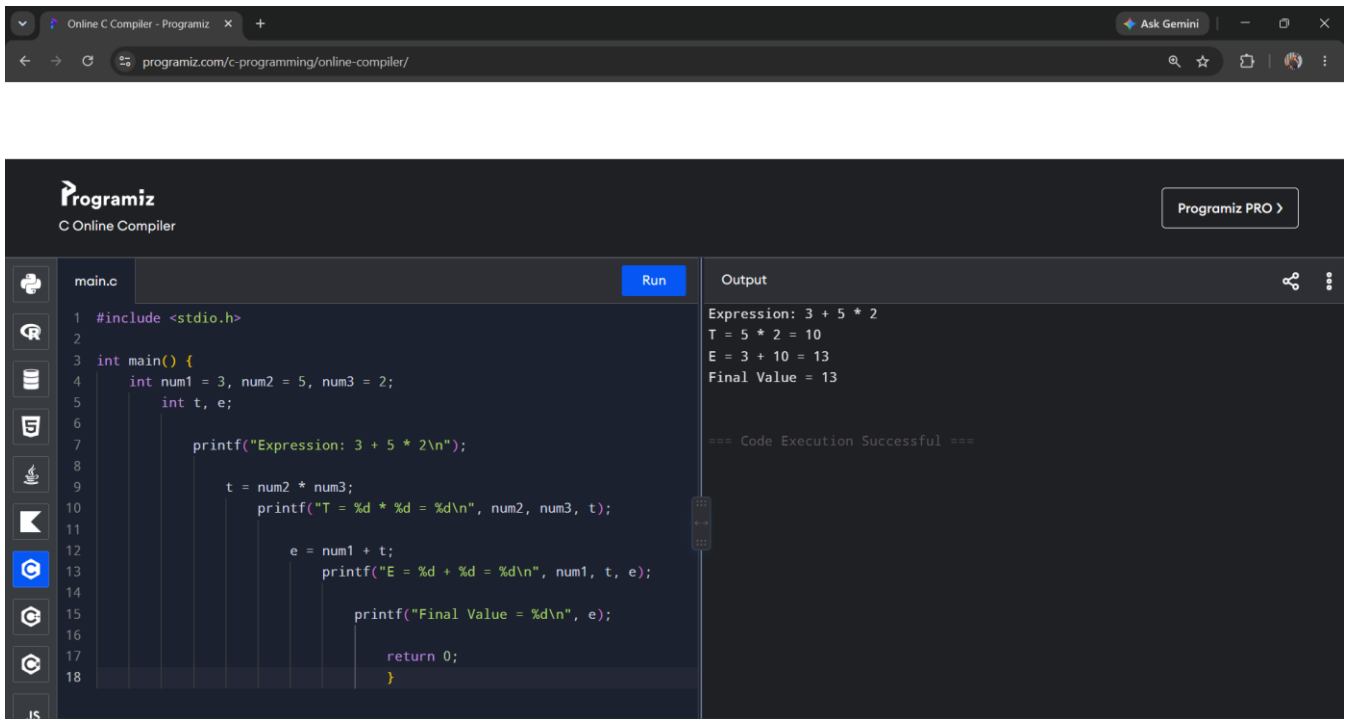
```
e = num1 + t;
```

```
printf("E = %d + %d = %d\n", num1, t, e);
```

```
printf("Final Value = %d\n", e);
```

```
return 0;
```

```
}
```



The screenshot shows the Programiz Online C Compiler interface. The left pane contains a C program in a file named `main.c`. The code is as follows:

```
1 #include <stdio.h>
2
3 int main() {
4     int num1 = 3, num2 = 5, num3 = 2;
5     int t, e;
6
7     printf("Expression: 3 + 5 * 2\n");
8
9     t = num2 * num3;
10    printf("T = %d * %d = %d\n", num2, num3, t);
11
12    e = num1 + t;
13    printf("E = %d + %d = %d\n", num1, t, e);
14
15    printf("Final Value = %d\n", e);
16
17    return 0;
18 }
```

The right pane, titled "Output", shows the execution results:

```
Expression: 3 + 5 * 2
T = 5 * 2 = 10
E = 3 + 10 = 13
Final Value = 13

=== Code Execution Successful ===
```

Result

Thus, the arithmetic expression $3 + 5 * 2$ was evaluated successfully and the final value is **13**.

2 Simple Type Checker

Aim

To implement a simple type checker for arithmetic expressions using the given symbol table.

Algorithm

1. Store the types of a, b, and c.

2. Check the type of $b * c$.
3. Since c is float, $b * c$ becomes float.
4. Check the type of $a + b * c$.
5. Display the type of each subexpression.

C Program

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main() {
```

```
    char a[] = "int";
```

```
    char b[] = "int";
```

```
    char c[] = "float";
```

```
    char type_bc[10];
```

```
    char type_expr[10];
```

```
    printf("a : %s\n", a);
```

```
    printf("b : %s\n", b);
```

```
printf("c : %s\n\n", c);
```

```
if (strcmp(b, "float") == 0 || strcmp(c, "float") == 0)
```

```
    strcpy(type_bc, "float");
```

```
else
```

```
    strcpy(type_bc, "int");
```

```
printf("Type of b * c = %s\n", type_bc);
```

```
if (strcmp(a, "float") == 0 || strcmp(type_bc, "float") ==  
0)
```

```
    strcpy(type_expr, "float");
```

```
else
```

```
    strcpy(type_expr, "int");
```

```
printf("Type of a + b * c = %s\n", type_expr);
```

```
return 0;
```

```
}
```

```
Output

a : int
b : int
c : float

Type of b * c = float
Type of a + b * c = float

=== Code Execution Successful ===
```

Result

Thus, the type of $b * c$ and the final expression $a + b * c$ was determined successfully as **float**. The symbol table in the question specifies $a:\text{int}$, $b:\text{int}$, and $c:\text{float}$.

3 Type Expression Equivalence

Aim

To determine whether two type expressions are equivalent.

Algorithm

1. Store the first type expression.
2. Store the second type expression.
3. Compare both expressions.
4. If they are same, display Equivalent.
5. Otherwise, display Not Equivalent.

C Program

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main() {
```

```
    char T1[] = "array(10, int)";
```

```
    char T2[] = "array(10, int)";
```

```
    printf("T1 = %s\n", T1);
```

```
    printf("T2 = %s\n\n", T2);
```

```
    if (strcmp(T1, T2) == 0)
```

```
        printf("Equivalent\n");
```

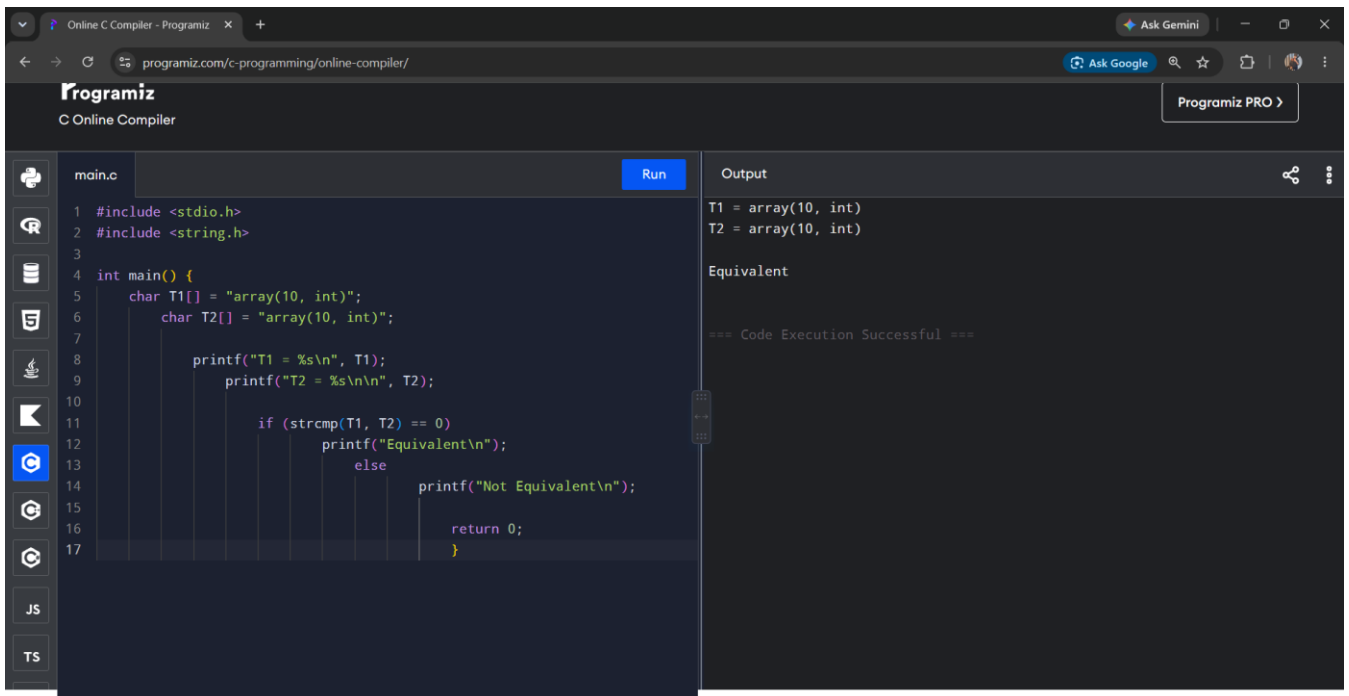
```
    else
```



```
printf("Not Equivalent\n");
```

```
return 0;
```

```
}
```



The screenshot shows the Programiz Online C Compiler interface. The code editor on the left contains the following C program:

```
1 #include <stdio.h>
2 #include <string.h>
3
4 int main() {
5     char T1[] = "array(10, int)";
6     char T2[] = "array(10, int)";
7
8     printf("T1 = %s\n", T1);
9     printf("T2 = %s\n", T2);
10
11     if (strcmp(T1, T2) == 0)
12         printf("Equivalent\n");
13     else
14         printf("Not Equivalent\n");
15
16     return 0;
17 }
```

The 'Run' button is highlighted. The output panel on the right shows the following text:

```
T1 = array(10, int)
T2 = array(10, int)

Equivalent

=== Code Execution Successful ===
```

Result

Thus, the two given type expressions are compared successfully and they are **Equivalent**. The question gives both expressions as `array(10, int)`.

4 Syntax Tree Construction

Aim

To construct and display the syntax tree for the expression $a + b * c$.

Algorithm

1. Create a node for +.
2. Create a as the left child.
3. Create * as the right child.
4. Create b and c as children of *.
5. Display the syntax tree.

C Program

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct Node {
```

```
    char data;
```

```
    struct Node *left;
```

```
    struct Node *right;
```

```
};
```

```
struct Node* createNode(char data) {
```

```
struct Node* newNode = malloc(sizeof(struct Node));
```

```
newNode->data = data;
```

```
newNode->left = NULL;
```

```
newNode->right = NULL;
```

```
return newNode;
```

```
}
```

```
void display(struct Node* root, int space) {
```

```
    if (root == NULL)
```

```
        return;
```

```
    space += 5;
```

```
    display(root->right, space);
```

```
    printf("\n");
```

```
    for (int i = 5; i < space; i++)
```

```
        printf(" ");
```

```
    printf("%c\n", root->data);
```

```
    display(root->left, space);

}

int main() {

    struct Node* root = createNode('+');

    root->left = createNode('a');

    root->right = createNode('*');

    root->right->left = createNode('b');

    root->right->right = createNode('c');

    printf("Syntax Tree for a + b * c:\n");

    display(root, 0);

    return 0;

}
```

```
main.c Run
4 struct Node {
5     char data;
6     struct Node *left;
10     struct Node* createNode(char data) {
11         struct Node* newNode = malloc(sizeof(struct Node));
12
42         root->left = createNode('a');
43
44         root->right = createNode('*');
45         root->right->left = createNode('b');
46         root->right->right = createNode('c');
47
48         printf("Syntax Tree for a + b * c:\n");
49
50 }
```

Output

Syntax Tree for a + b * c:

```

      c
     *
    b
   +
  a

```

=== Code Execution Successful ===

Result

Thus, the syntax tree for the expression $a + b * c$ was constructed and displayed successfully. The question specifies this expression and grammar.

5 Inherited and Synthesized Attributes

Aim

To demonstrate the propagation of inherited and synthesized attributes for a variable declaration.

Algorithm

1. Set the type as int.
2. Read the identifiers a, b, and c.
3. Pass the type int to each identifier.

4. Display the type assigned to every identifier.

C Program

```
#include <stdio.h>
```

```
int main() {
```

```
    char type[] = "int";
```

```
    char id1[] = "a";
```

```
    char id2[] = "b";
```

```
    char id3[] = "c";
```

```
    printf("Declaration: int a,b,c\n\n");
```

```
    printf("%s : %s\n", id1, type);
```

```
    printf("%s : %s\n", id2, type);
```

```
    printf("%s : %s\n", id3, type);
```

```
    return 0;
```

```
}
```

Programiz

C Online Compiler

Programiz PRO >

main.c

Run

```
1 #include <stdio.h>
2
3 int main() {
4
5     char type[] = "int";
6
7     char id1[] = "a";
8     char id2[] = "b";
9     char id3[] = "c";
10
11     printf("Declaration: int a,b,c\n\n");
12
13     printf("%s : %s\n", id1, type);
14     printf("%s : %s\n", id2, type);
15     printf("%s : %s\n", id3, type);
16
17     return 0;
18 }
```

Output

Declaration: int a,b,c

a : int

b : int

c : int

=== Code Execution Successful ===

Result

Thus, the type `int` was successfully propagated and assigned to the identifiers **a**, **b**, and **c**. The question gives the declaration `int a,b,c` and asks to display the type assigned to each identifier.